



Final Year Project Proposal

Title	Buzzly: A Centralized Hub for Digital Marketing Intelligence
Student Name	Pachara Danthanin
Student Code	66313319
Semester / Year	2 / 2025
Degree	B.Sc. Computer Science
Advisor	Asst. Prof. Dr. Wansuree Massagram
Committee	Asst. Prof. Dr. Sanya Khruahong Asst. Prof. Dr. Winai Wongthai

Presented in fulfillment of course: 254491 Seminar

Department of Computer Science and Information Technology
Faculty of Science, Naresuan University

Title

Buzzly: A Centralized Hub for Digital Marketing Intelligence

Abstract

This research presents the development of "Buzzly," a centralized marketing intelligence platform designed to address the challenges of data fragmentation and the lack of unified analytical tools. By consolidating cross-channel campaign tracking into a single ecosystem, Buzzly aims to optimize marketing workflows and empower data-driven strategic decision-making.

The system is built as a Single Page Application (SPA) using React.js and TypeScript, integrated with a Supabase Backend-as-a-Service (BaaS) architecture. This approach leverages a robust PostgreSQL database to ensure scalable data processing and secure, real-time API synchronization. Key functionalities include standardized metric comparisons and granular performance analytics across multiple marketing platforms.

To ensure system reliability and effectiveness, a rigorous evaluation framework was applied: Lighthouse for performance auditing, k6 for load testing, and Data Consistency Checks. Finally, the platform's practical utility and user satisfaction were validated using the System Usability Scale (SUS).

Chapter 1

Introduction

1.1 Introduction

In the contemporary digital marketing landscape, organizations rely on multi-channel advertising to maximize reach. However, this inevitably causes data fragmentation, where critical marketing metrics are siloed across diverse platforms (e.g., Meta Ads, Google Ads). Marketing practitioners thus face significant cognitive overload when manually extracting, normalizing, and comparing Key Performance Indicators (KPIs) to evaluate true advertising effectiveness. The absence of a centralized analytical tool impedes data-driven strategic decision-making. To resolve this, this project proposes "Buzzly", an integrated marketing intelligence platform leveraging a Backend-as-a-Service (BaaS) architecture to unify cross-channel data into a centralized perspective.

1.2 Objectives

- 1) To design and develop "Buzzly," a centralized marketing analytics platform utilizing React.js and Supabase.
- 2) To engineer automated API pipelines that periodically extract and normalize fragmented data from external advertising platforms.
- 3) To construct a responsive User Interface (UI) that applies progressive disclosure techniques to minimize cognitive load during data analysis.

1.3 Scope

Target Users: Digital marketers, campaign managers, and enterprise organizations.

Core Functionalities:

- Multi-tenant authentication and role-based access control (RBAC).
- Scheduled data synchronization via third-party advertising APIs.
- A comprehensive dashboard for visualizing cross-platform metrics.
- Metric Standardization (MVP Scope): The Unified Data Dictionary (UDD) mapping is strictly bounded to five foundational Key

Performance Indicators (KPIs): Impressions, Clicks, CTR, CPC, and Ad Spend.

- Contingency Ingestion: A deterministic fallback mechanism supporting manual flat-file payloads (.csv, .xlsx) to maintain data continuity.

Technical Constraints & Quantitative Benchmarks:

- Developed as a Single Page Application (SPA) using React.js (TypeScript) and Supabase (PostgreSQL).
- Success Metrics: System validation requires achieving a Google Lighthouse performance score of $\geq 80/100$, sustaining stable response times for 50 concurrent users (simulating a standard enterprise marketing team) via k6 load testing, and attaining a System Usability Scale (SUS) score of ≥ 68 (industry standard for acceptable usability).

1.4 Definition of Terms

Data Fragmentation: The scattering of marketing data across isolated third-party advertising platforms, making aggregate analysis difficult.

Progressive Disclosure: A UI/UX design technique that sequences information and actions across multiple views to reduce a user's cognitive load.

Backend-as-a-Service (BaaS): A cloud computing model (specifically Supabase in this project) that provides centralized database management and authentication without requiring a traditional server infrastructure.

Customer 360: A comprehensive, unified view of aggregated marketing performance and metrics from multiple platforms.

1.5 Expected Benefits

- 1) **Operational Efficiency:** Eliminates manual data aggregation, significantly reducing campaign reporting time.
- 2) **Enhanced Decision-Making:** Provides a comprehensive overview of marketing performance to optimize advertising budgets.
- 3) **Scalable Architecture:** Establishes a highly scalable database foundation capable of supporting future integrations.

Chapter 2

Literature Review

In this project, we derive concepts and methodologies from related research studies and existing products, emphasizing the development of marketing web applications. Various approaches have been explored, including the application of modern web technologies, database architecture, and User Interface/User Experience (UX/UI) design to enhance system efficiency and user engagement. The following research studies have played a crucial role in shaping our methodology, providing insights into different techniques for developing marketing platforms. By analyzing these works, we aim to refine our approach and develop an efficient and practical solution.

2.1 Digital Marketing Concepts

As digital marketing evolves, managing multi-channel campaigns requires specialized tools. A primary challenge, however, is data fragmentation. Customer information often remains siloed across different platforms, leading to inconsistent data formats and misaligned sales and marketing strategies [1, 2]. These technical bottlenecks hinder the creation of a unified customer profile and restrict consistent personalization.

To overcome these challenges, a centralized data ecosystem is essential. Cloud-native architectures that integrate a Customer Data Platform (CDP) or CRM serve as the foundation for a "Customer 360" view [1, 2]. By utilizing real-time APIs, these systems enable seamless data orchestration, dynamic segmentation, and data-driven decision-making [2, 3].

Ultimately, centralized data resolves critical measurement challenges. Without a unified structure, tracking Key Performance Indicators (KPIs) across various touchpoints is highly inaccurate [1]. Centralization not only ensures consistent KPI tracking but also supports strict auditability for data governance [3]. By addressing these barriers, the proposed platform aims to optimize marketing workflows and enhance strategic execution.

2.2 UI/UX Design and Web Development

Following the establishment of marketing requirements, the design and user experience of the platform must be carefully addressed. Effective User Interface (UI) and User Experience (UX) design are critical for ensuring user engagement and operational efficiency in web applications. In the context of marketing intelligence platforms, users are frequently exposed to dense and complex datasets, making cognitive overload a primary concern. To mitigate this, literature emphasizes the application of analytics-based visualizations. Transforming abstract data into highly understandable visual elements significantly extends human cognitive abilities and reduces information

overload [4]. Furthermore, implementing progressive disclosure—where detailed analytical data is revealed only on demand—helps decision-makers readily identify patterns without unnecessary visual clutter [4, 5].

Developing these intuitive interfaces requires a rigorous User-Centered Design (UCD) methodology. Research indicates that incorporating iterative heuristic evaluations and usability testing ensures the dashboard aligns with business management goals and user expectations [5]. Additionally, for web-based applications, utilizing continuous evaluation methods such as A/B testing is crucial. Systematic A/B testing allows development teams to validate layout configurations, measure the impact of UI changes, and maximize overall user satisfaction based on data-driven insights [6]. By integrating these foundational UI/UX principles, the proposed system can minimize cognitive friction, guide user attention efficiently, and ultimately accelerate decision-making.

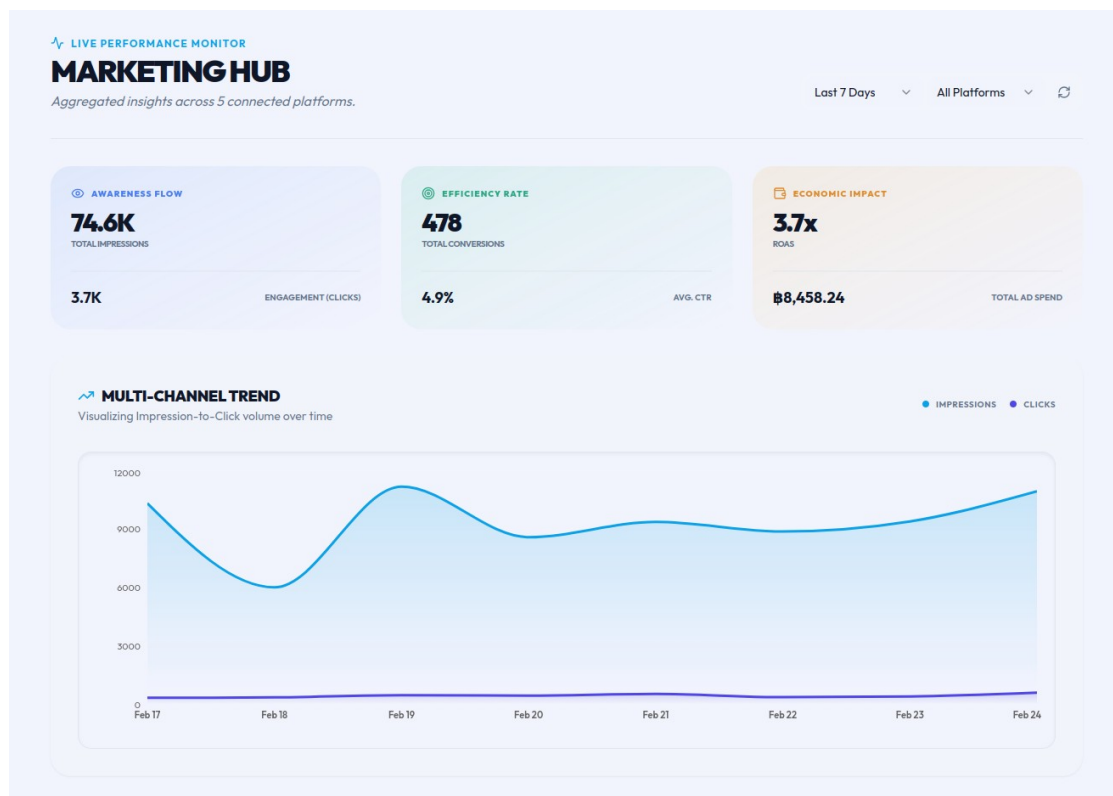


Fig. 2.1. The conceptual interface of Buzzly, demonstrating the integration of marketing data and progressive disclosure. *[Author's original work]*

2.3 Technologies, Backend-as-a-Service (BaaS), and Tools

The robust functionality of a marketing web application relies heavily on its underlying architecture. This section explores the modern web technologies necessary for handling complex data relationships and maintaining system scalability. For the frontend interface, the project utilizes React.js, operating as a Single Page Application (SPA). Literature indicates that React's component-based architecture and virtual DOM manipulation provide highly responsive user experiences, which are essential for real-time data visualization and marketing dashboards [7].

Instead of developing a traditional server-side backend (e.g., Node.js with Express), this project adopts a Backend-as-a-Service (BaaS) architecture using Supabase. BaaS platforms drastically reduce development overhead by providing out-of-the-box backend functionalities, allowing developers to focus entirely on frontend user experiences [8]. Within this architecture, the React frontend communicates directly with a centralized PostgreSQL database via secure, auto-generated APIs. To ensure data security and user authorization without a traditional middleware layer, Supabase implements rigorous Row Level Security (RLS) policies directly at the database level. This modern serverless-like ecosystem enhances data synchronization, accelerates the development lifecycle, and ensures reliable data integrity across the platform [9].

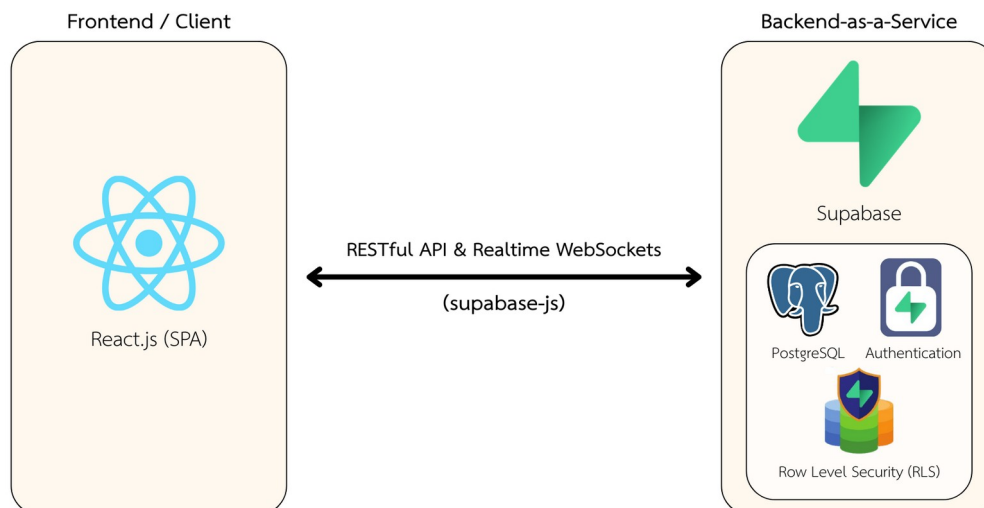


Fig. 2.2. Proposed system architecture for Buzzly, featuring React.js (SPA) and Supabase as a Backend-as-a-Service (BaaS) for real-time data integration. *[Author's original work]*

2.4 Related Works and Existing Systems

While existing literature proposes various approaches to marketing web applications, significant limitations in integration and usability remain. Although omnichannel systems attempt to unify customer touchpoints, their implementation is often hindered by fragmented data layers and unresolved informational challenges [10].

Similarly, efforts utilizing SaaS frameworks and cross-functional data marts frequently suffer from persistent data silos, uneven data quality, and semantic discrepancies between marketing tools [11, 12]. These inconsistencies severely limit the synthesis of reliable cross-channel analytics and high-level KPIs.

Addressing this critical research gap, the proposed system, Buzzly, offers a scalable, centralized hub designed to overcome these integration barriers. By establishing standardized cross-platform metrics and operational governance, Buzzly ensures data consistency and empowers practitioners with trustworthy marketing intelligence.

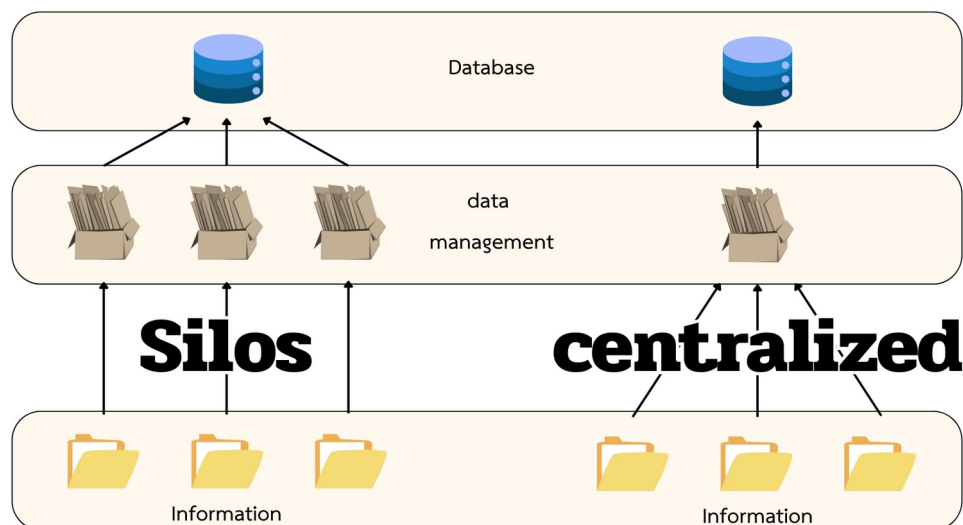


Fig. 2.3. Comparison between fragmented marketing data siloed and the Buzzly centralized intelligence hub. *[Author's original work]*

2.5 Data Privacy and Regulatory Compliance

The aggregation of cross-channel marketing data necessitates strict adherence to data protection frameworks, notably the General Data Protection Regulation (GDPR) and the Personal Data Protection Act (PDPA) of Thailand. Within the operational context of a multi-tenant marketing intelligence platform, the system functions as a *Data Processor*, while the client organizations act as *Data Controllers*.

To ensure regulatory compliance, the proposed architecture integrates "Privacy by Design" principles, embedding data-minimizing controls into the service lifecycle [14]. This involves the systematic anonymization of Personally Identifiable Information (PII) prior to database insertion. Furthermore, strict data isolation is enforced at the database level using PostgreSQL Row Level Security (RLS) to prevent unauthorized cross-tenant data access, a critical requirement for scalable SaaS architectures [15]. For automated data ingestion, the platform employs secure delegated authorization protocols (e.g., OAuth 2.0), ensuring that external data extraction strictly aligns with the granular consent scopes authorized by the Data Controller [16].

Chapter 3

Methodology

3.1 Software Development Methodology

This project adopts the Prototyping Model, an iterative methodology suitable for UI-intensive web applications. The development lifecycle consists of four main phases:

1. **Requirement and API Analysis:** Defining system requirements, marketing KPIs, and analyzing third-party API data structures (e.g., Meta and Google Ads) to design a unified database schema.
2. **UI/UX Prototyping:** Designing system interfaces and user workflows. This phase focuses on applying progressive disclosure principles and creating modular component layouts for the React.js frontend.
3. **System Implementation:** Developing the React.js Single Page Application (SPA) and configuring the Supabase backend. This includes writing Edge Functions for the on-demand ETL pipeline and implementing Row Level Security (RLS) for multi-tenant access control.
4. **Testing and Evaluation:** Evaluating system performance using k6 (load testing) and Google Lighthouse (frontend auditing). User satisfaction is subsequently measured using the System Usability Scale (SUS).

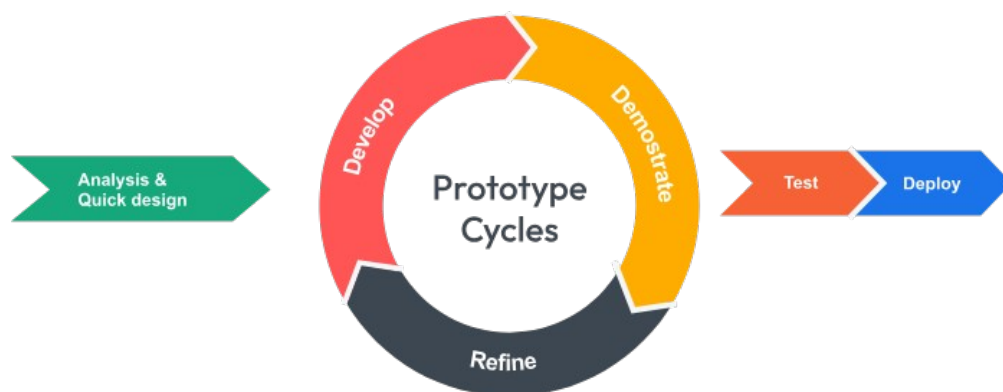


Fig. 3.1. Prototyping Model. [13]

3.2 Hardware and Software Requirements

The implementation, testing, and deployment of the proposed system necessitate specific hardware and software configurations. The selected technology stack aligns with modern web development standards and AI-assisted workflows.

3.2.1 Hardware Requirements

Development Environment: A standard workstation equipped with a multi-core processor and sufficient memory (minimum 8GB RAM) to execute local development servers and load-testing scripts.

Deployment Infrastructure: A cloud-based edge hosting platform (e.g., Vercel or Netlify) for deploying and serving the frontend web application.

3.2.2 Software Requirements

Development Tools & CI/CD Pipeline:

Antigravity: An AI-powered Integrated Development Environment (IDE) utilized for code generation, logic management, and system integration.

Git and GitHub Actions: Employed for version control and establishing a Continuous Integration/Continuous Deployment (CI/CD) pipeline to automate testing and seamlessly deploy the React SPA to edge hosting platforms (e.g., Vercel/Netlify).

UI/UX Design:

Lovable: An AI-driven design platform used for rapid prototyping and automated user interface generation.

Frontend Technologies:

React.js (via Vite): The core JavaScript library, written in TypeScript, utilized to construct the Single Page Application (SPA).

Backend Infrastructure & Cloud Budgeting:

Supabase: A Backend-as-a-Service (BaaS) platform integrated to manage the PostgreSQL relational database, user authentication, and Row Level Security (RLS).

Infrastructure Strategy: To ensure financial viability, the architecture is strictly optimized to operate within cloud-native Free/Hobby tiers during the 8-month prototyping phase, dynamically allocating memory-

intensive ETL tasks (Apache Airflow) to local environments or highly constrained cloud instances.

Testing and Evaluation:

Lighthouse: An automated tool utilized for performance and accessibility auditing.

k6: An open-source testing tool employed for system load and stress testing.

3.3 Work Schedule

The project execution spans approximately eight months, aligned with the prototyping iterations. The structured timeline is outlined in Table 3.1.

Table 3.1: *Project Work Schedule*

No.	Activity	Mar.	Apr.	May	Jun.	Jul.	Aug.	Sept.	Oct.
1	Requirements Analysis & PRD Documentation	↔							
2	System Flow & UI/UX Design (Swim Lane & Lovable)		↔						
3	Database Schema & Architecture Design		↔						
4	System Implementation (React & Supabase via Antigravity)				↔				
5	System Testing & Performance Benchmarking (k6 & SUS)						↔		
6	Final Report Documentation & Project Presentation								↔

3.4 System Requirements and Initial Design

To establish the functional and structural boundaries of the Buzzly platform, a comprehensive system design was formulated. This section outlines the core use cases, user interface strategies, and the relational data architecture required for enterprise-level marketing intelligence.

3.4.1 Functional Requirements and Use Cases

The system provides three core functional modules to support multi-channel marketing analytics:

Identity and Access Management: Utilizes Supabase Auth to issue JSON Web Tokens (JWT) and enforces Role-Based Access Control (RBAC) for multi-tenant workspace isolation.

On-Demand Data Ingestion: Executes event-driven RESTful API requests to retrieve JSON payloads from external advertising endpoints (e.g., Meta Graph API, Google Ads API) upon client initiation.

Metric Standardization: Implements a Unified Data Dictionary (UDD) to parse and map disparate vendor-specific data structures (e.g., Meta's 'outbound_clicks' vs. Google's 'clicks') into a Common Data Model (CDM), enabling the accurate and consistent computation of aggregated Key Performance Indicators (KPIs).

The interactions between the end-user (Actor) and these system modules are defined in the Use Case Diagram (**Figure 3.2**).

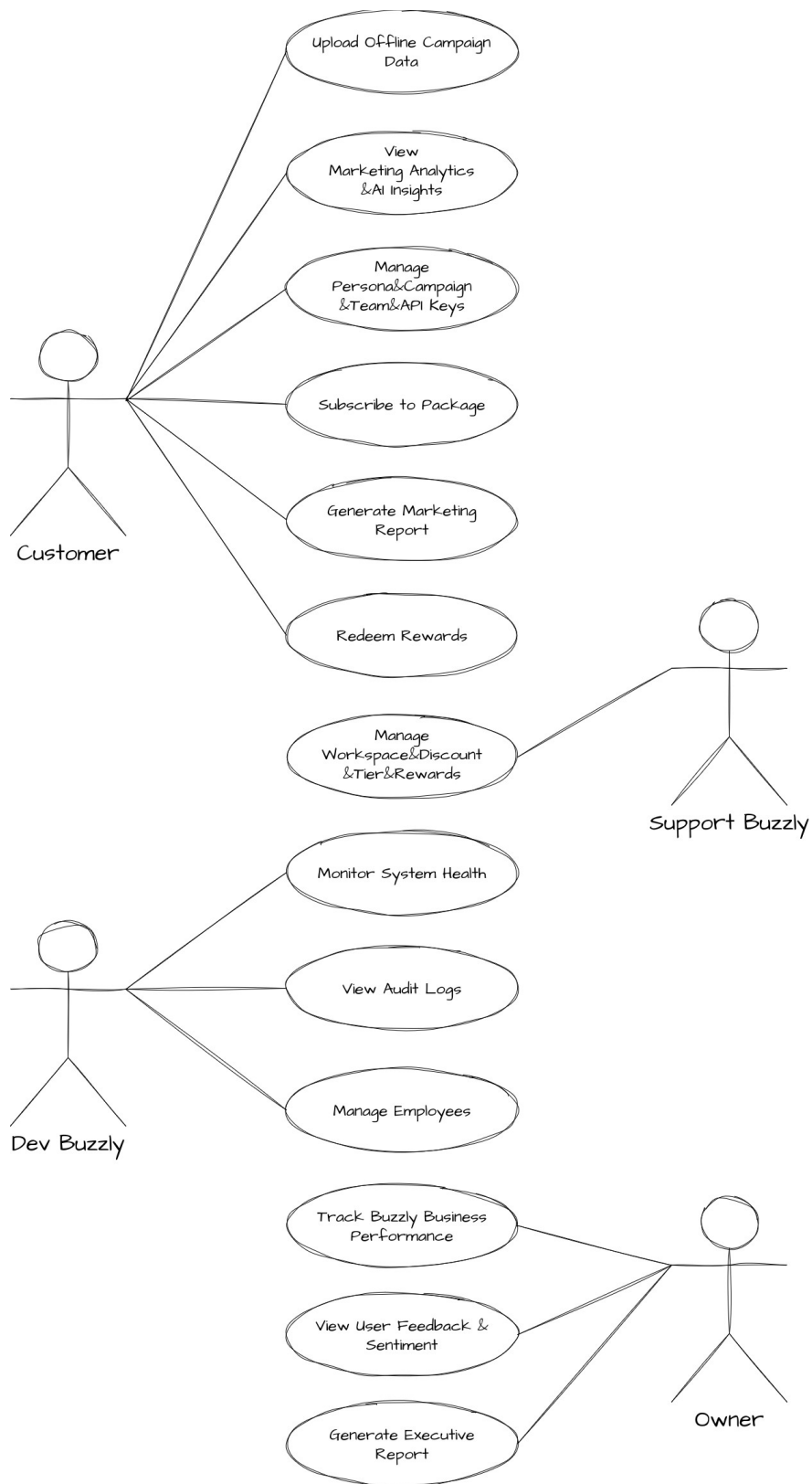


Fig. 3.2. Use Case diagram illustrating the core interactions within the Buzzly platform.
[Author's original work]

3.4.2 User Workflow Analysis

The platform operates on an event-driven model, restricting data synchronization to explicit client-side triggers. Figure 3.3 presents the system's operational sequence via a swimlane diagram. This workflow delineates the critical path of the on-demand synchronization process: a user-initiated event initiates frontend request validation, executes the backend ETL pipeline, and asynchronously updates the client interface to reflect the newly normalized dataset.

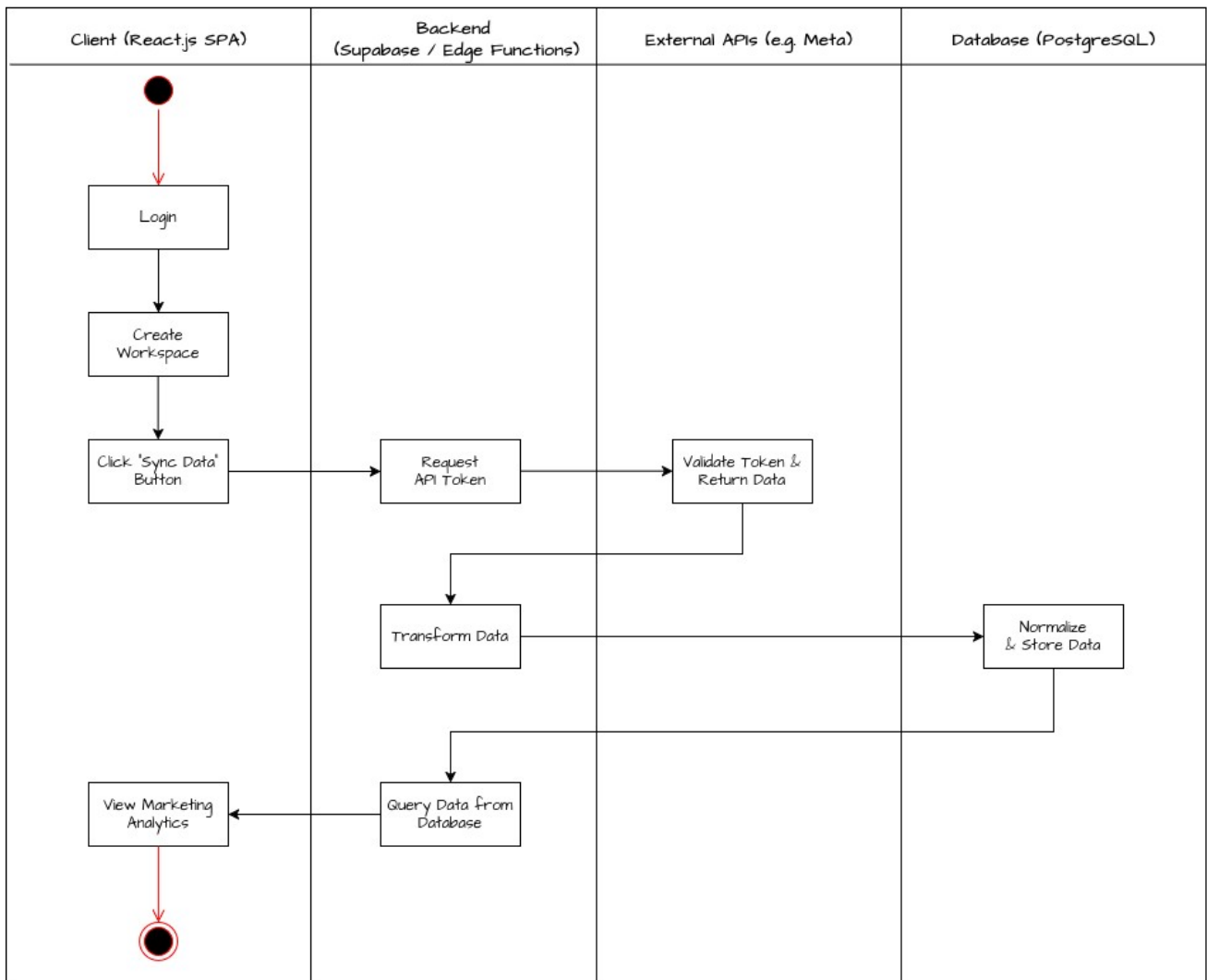


Fig. 3.3. User Workflow Swimlane Diagram. *[Author's original work]*

3.4.3 On-Demand System Architecture and Data Flow

The system utilizes an event-driven Extract, Transform, Load (ETL) architecture to manage third-party API rate limits and optimize server execution. Figure 3.4 details the data ingestion pipeline through a synchronous six-step sequence:

[1-2] Invocation: A client-side trigger **(1)** prompts the React.js frontend to dispatch an HTTP POST request **(2)** to the Supabase serverless functions.

[3-4] Extraction: The backend functions transmit data requests **(3)** to external advertising APIs (e.g., Meta Graph API). The external endpoints return the unformatted marketing metrics as JSON payloads **(4)**.

[5] Transformation & Load: The serverless functions parse and normalize the JSON structures into a unified schema, executing write operations to store the processed records in the PostgreSQL database **(5)**.

[6] Retrieval: The React.js Single Page Application (SPA) queries the database to fetch the updated, normalized metrics for interface rendering **(6)**.

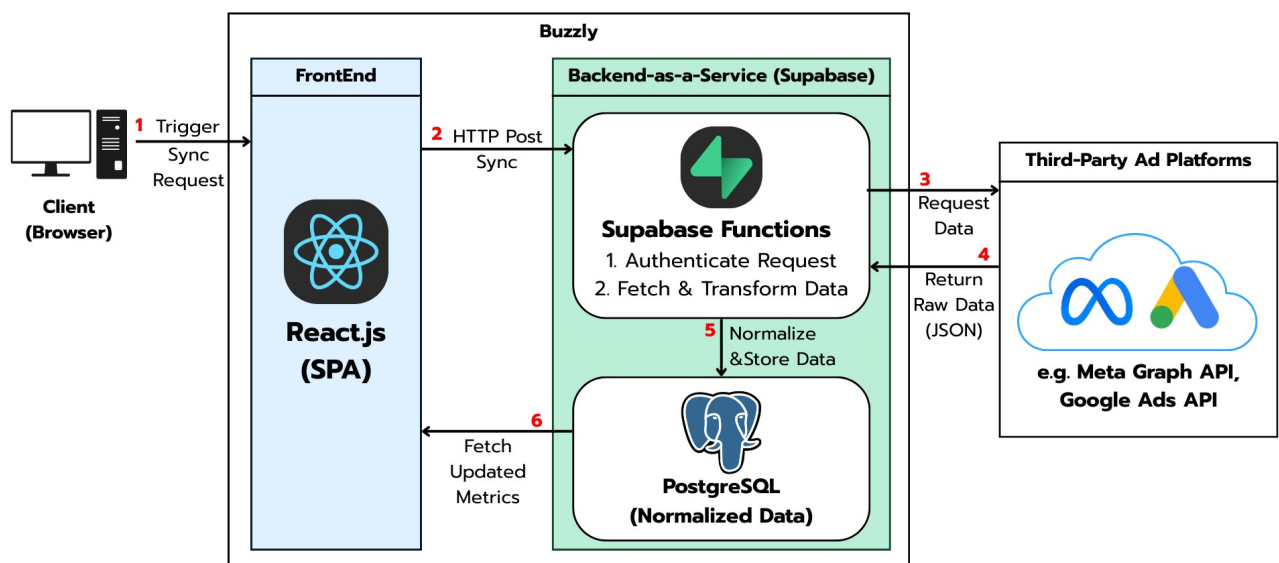


Fig. 3.4. On-demand system block diagram illustrating the event-driven ETL data pipeline.
[Author's original work]

3.4.4 Initial UX/UI Design and Prototyping

To minimize cognitive load, the User Interface (UI) applies progressive disclosure techniques. Utilizing AI-assisted design platforms (Lovable), a highly responsive Single Page Application (SPA) interface was prototyped. Figure 3.3 demonstrates the primary "Customer 360" dashboard, prioritizing high-level KPIs.

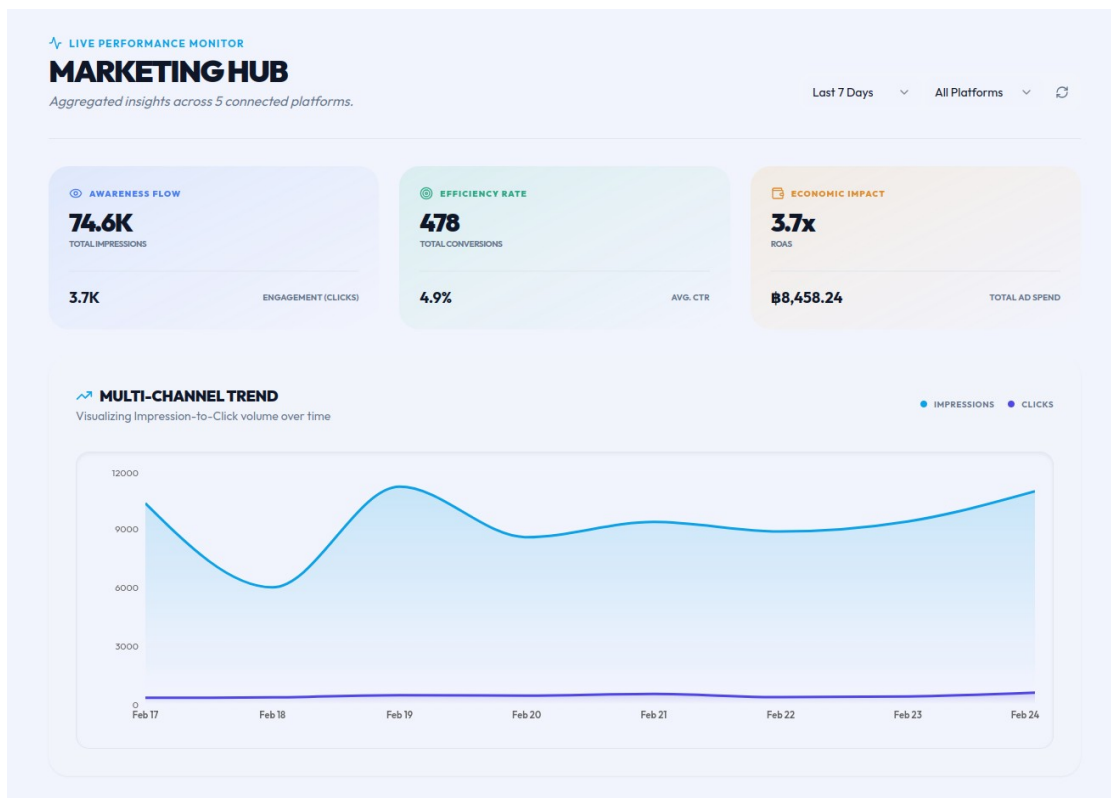


Fig. 3.5. Buzzly main dashboard. *[Author's original work]*

3.4.5 Conceptual Entity-Relationship (ER) Database Design

The system implements a relational database architecture (PostgreSQL) strictly normalized to the Third Normal Form (3NF) to enforce data integrity across multi-tenant and time-series datasets. To manage schema complexity, the data model is logically partitioned via Domain-Driven Design (DDD) into three bounded contexts: Identity Management, API Integration, and Unified Analytics. The Unified Analytics domain operates strictly on a Common Data Model (CDM) governed by the platform's Unified Data Dictionary, ensuring semantic consistency across all integrated data sources. Figure 3.5 presents the conceptual Entity-Relationship (ER) model, defining the referential dependencies and cardinalities among the core business entities.

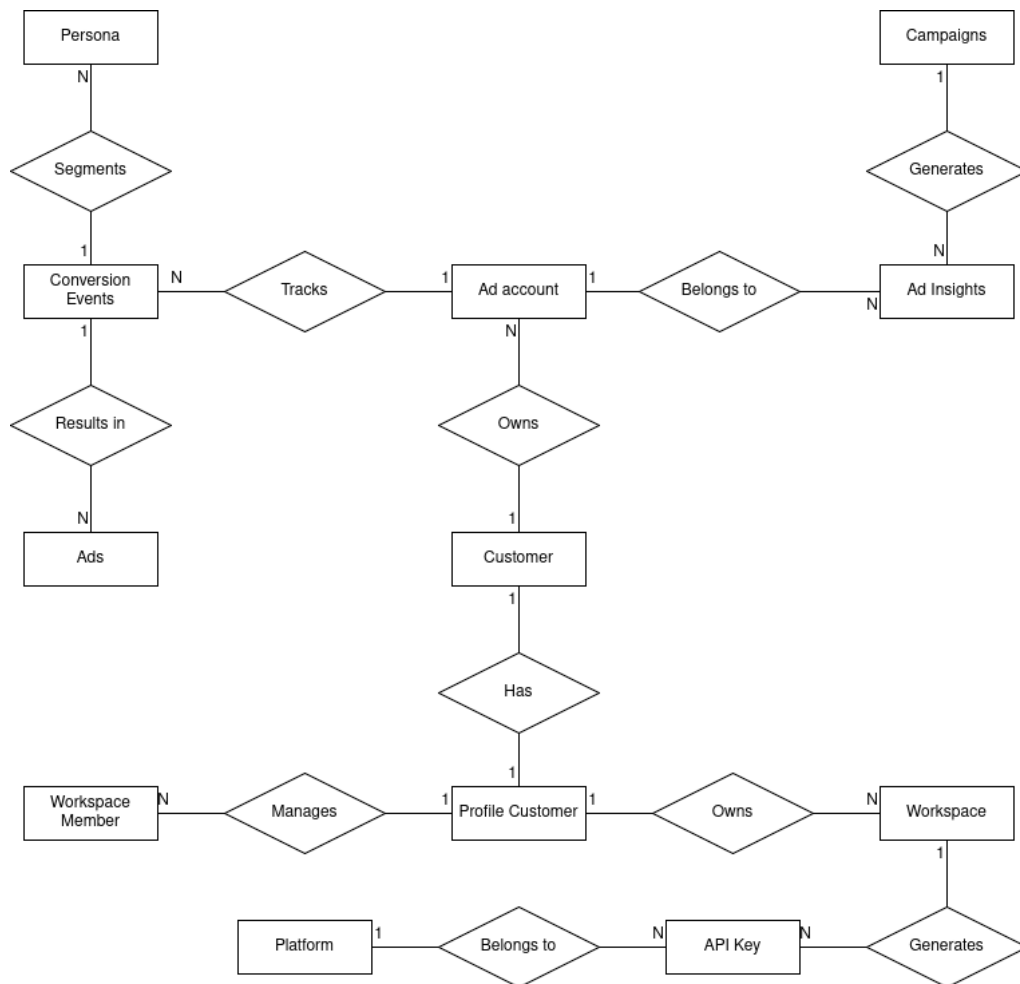


Fig. 3.6. Conceptual Entity-Relationship (ER) model defining the referential dependencies and cardinalities of the core bounded contexts. *[Author's original work]*

3.4.6 System Resilience and Fallback Ingestion

To mitigate external API failures (e.g., rate limiting, timeouts), the platform implements a two-tier hybrid fallback mechanism for manual data ingestion via flat-file payloads (.csv, .xlsx). The architecture initiates with a synchronous client-side pre-validation layer within the React.js frontend, executing heuristic schema checks to enforce header integrity and primitive data type constraints. This layer provides low-latency error resolution, preventing foundational structural violations from entering the data ecosystem.

Valid payloads are temporarily staged in a secure cloud bucket, initiating an asynchronous Apache Airflow ETL pipeline. Python-based tasks utilizing the Pandas library execute automated data remediation—dynamically resolving schema misalignments and normalizing temporal data—ensuring strict compliance with the platform's Common Data Model (CDM) prior to PostgreSQL database insertion via Psycpg2. To manage unrecoverable anomalies, the pipeline routes fatal exceptions to a Dead Letter Queue (DLQ) and registers diagnostic metadata into a PostgreSQL telemetry table. The React.js client subscribes to this table via Supabase Realtime WebSockets, surfacing immediate, non-blocking diagnostic alerts to the UI, thereby closing the asynchronous feedback loop while preserving operational continuity.

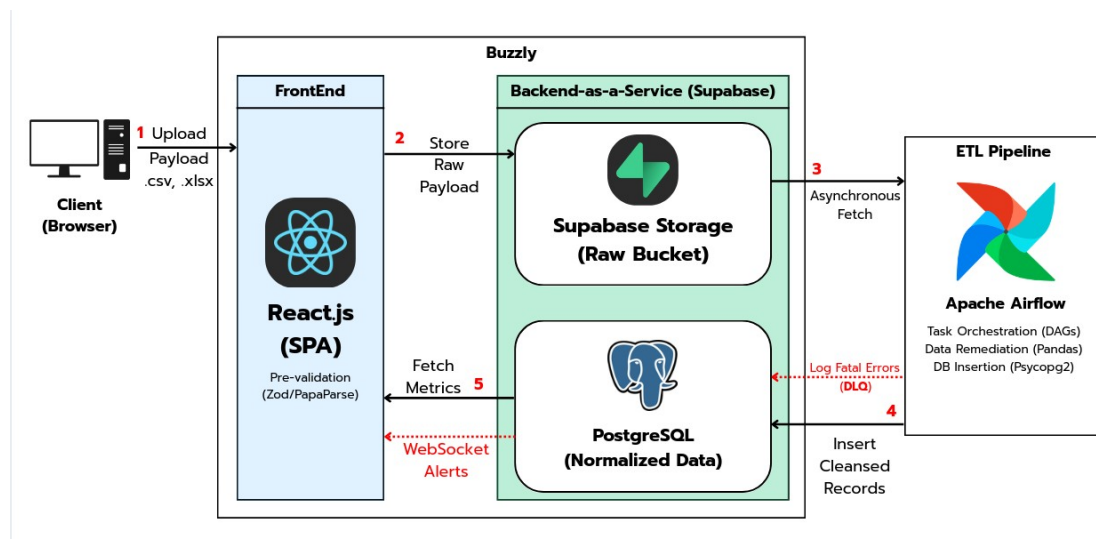


Fig. 3.7. Asynchronous data fallback ingestion pipeline illustrating the integration of React.js, Supabase Storage, and **Apache Airflow**, including DLQ telemetry and WebSocket alerts for automated data remediation.*[Author's original work]*

3.5 Project Risk Management and Scope Fallback

Given the strict 8-month academic timeline, the project incorporates a deterministic feature fallback strategy to mitigate technical deployment risks. If the asynchronous **Apache Airflow** ETL pipeline, **Pandas remediation tasks**, or complex Supabase Edge Functions encounter unresolvable integration delays, the development scope will dynamically pivot to a "Thin-Backend" contingency model. This involves temporarily bypassing **Airflow** and routing pre-validated .csv payloads directly from the React.js client to PostgreSQL via simplified Remote Procedure Calls (RPCs). This strategic fallback ensures the core "Customer 360" dashboard remains fully operational and minimum academic requirements are strictly met without catastrophic project failure.

References

- [1] Darvidou, K. (2024). Omnichannel Marketing in the Digital Age: Creating Consistent, Personalized and Connected Customer Experiences. *Technium Business and Management*, 1(1), 1-15.
- [2] Dash, S. (2025). Designing an Integrated Enterprise Architecture for Unified Sales and Marketing Operations: Enhancing Customer Experience through Data-Driven Decision-Making. *European Journal of Science, Innovation and Technology*, 5(3), 1-12.
- [3] Gupta, A. (2024). Marketing Automation Reinvented: AI-Powered Journey Orchestration Using Adobe Campaign on Cloud Platforms. *Indian Scientific Journal Of Research In Engineering And Management*, 8(3), 1-10.
- [4] Mikalef, P., et al. (2024). An experimental study on dashboard visualizations. *Information & Management*, 61(6), 104011.
- [5] Bennett, A., & Loose, S. M. (2024). User-Centered Development of an Online Dashboard Tool for Economic Sustainability for Small and Medium Enterprises. *Sustainability*, 16(2), 557.
- [6] Quin, F., Weyns, D., Galster, M., & Costa Silva, C. (2024). A/B testing: A systematic literature review. *Journal of Systems and Software*, 211, 112011.
- [7] Jonathan, R., & Supriyadi. (2023). Development of front-end web applications utilizing single page application framework and react.js library. *International Journal of Software Engineering and Computer Science (IJSECS)*, 3(3), 529-536.
- [8] Okanda, P., Chhatbar, A., & Njeru, O. (2024). DbAPI: A Backend-as-a-Service Platform for Rapid Deployment of Cloud Services. *2024 IST-Africa Conference (IST-Africa)*, IEEE, 1-12.
- [9] Ayezabu, A. (2022). Supabase vs Firebase: Evaluation of performance and development. *Metropolia University of Applied Sciences*.
- [10] Cui, T. H., et al. (2021). Informational Challenges in Omnichannel Marketing: Remedies and Future Research. *Journal of Marketing*, 85(1), 103–120.
- [11] Patel, P. (2024). Cross-functional data modeling and data mart design for sales intelligence. *ResearchGate*.

- [12] Subham, K. (2025). Scalable SaaS Implementation Governance for Enterprise Sales Operations. *International journal of computational and experimental science and engineering*, 11(3).
- [13] Prototyping Model. <https://www.bairesdev.com/methodologies/>
- [14] O. Oluoha, A. Odesina, and O. Reis, "A privacy-first framework for data protection and compliance assurance in digital ecosystems."
- [15] S. Pushpan, "Multi-Tenant Architecture: A Comprehensive Framework for Building Scalable SaaS Applications," *International journal of scientific research in computer science, engineering and information technology*, vol. 10, no. 6, pp. 1117–1126, Nov. 2024.
- [16] L. Ferretti, M. Marchetti, and M. Colajanni, "Verifiable Delegated Authorization for User-Centric Architectures and an OAuth2 Implementation," *IEEE COMPSAC*, vol. 2, pp. 718–723, July 2017.